

PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM (SRMS)

DONE BY



NAME: SHAIK ISMAIL
REGISTRATION NUMBER: AP24110011823

CSE 2ND YEAR THIRD SEMESTER

SECTION-W

SUBMITTED TO
RAKESH RAMA RAJU
CODING SKILLS – I (CSE 201)

ABSTRACT

- This project is a simple Student Record Management System (SRMS) created using the C programming language.
- The main idea is to maintain student details like roll number, name, and marks in a structured and safe way.
- The system works through a menu - driven interface, making it easy for any user to operate.
- It uses file handling, which means the data is stored permanently in text files, even after closing the program.

Two files are used:

credentials.txt → stores username, password, and role

student.s.txt → stores all student records

The program includes a login system with two roles:

Admin. → can add, update, delete, display, and search records

User → can only display and search records

This separation makes the system more secure and realistic.

The project helped me learn many concepts such as:

Reading and writing files

Using structures to group data

Breaking logic into smaller modules

Working with loops and conditions

The SRMS project improved my understanding of how real - world

management systems work.

- Overall, this project strengthened my programming skills and gave me practical experience with data handling.

INTRODUCTION

- Student data management is an important task in schools, colleges, and training centers.

- Traditional paper- based methods have many problems like:

- Difficulty in searching

- Possibility of losing records

- Misplacement of files

- Slow updating process

- This project solves these issues by using a simple computerized system built in C.

- The system is fully console - based and uses text files to store information.

- The program is divided into modules so that each task is easy to understand and maintain.

- The login system ensures that only authorized people can access sensitive operations.

- Admin has full control, while the user has limited access, making the system safer.

- Important C concepts used are:

- Structures

- Functions

- File handling

- Loops and conditions

- Switch - case menus

- The project shows how even basic C programming can create a working real- life application.

PROBLEM STATEMENT

- Many institutions still use manual or paper - based student record systems.
 - This leads to several problems:
 - Records can get damaged or lost
 - Searching for a student takes a long time
 - Updating or deleting information is difficult
 - Mistakes happen easily
 - No security or restricted access
 - Even simple Excel sheets can become confusing when data grows.
 - Correctness, safety, and quick access are big challenges in manual
 - - There is a need for a small and reliable program that can:
 - Store records properly
 - Update details safely
 - Search information easily
 - Delete unused records
 - Only allow approved users to make changes
 - The aim is to design a clean and simple system using C language.
- The system must ensure data safety using files and provide different access modes for admin and user.
- Therefore, the main problem addressed here is building a basic, secure, and efficient student record management tool.

OBJECTIVES

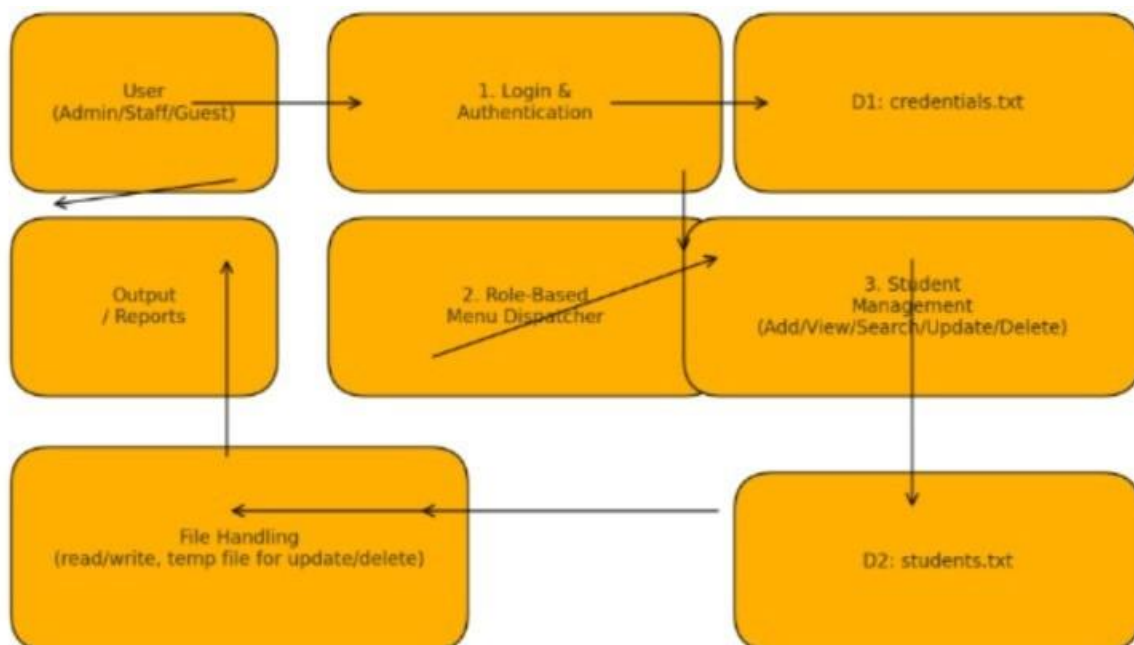
- To build a working Student Record Management System using C programming.
 - To store and manage student information in a proper and safe format.
 - To use structures for grouping student details.
 - To understand and practice file handling in real applications.
 - To create a login system using username, password, and role.
 - To separate the privileges of admin and user.
 - To develop modules such as:
 - Add student
 - Display records
 - Search student
 - Update records
 - Delete records
 - To learn how data is read from and written to text files.
 - To practice menu - driven programming using loops and switch cases.
 - To write clean, readable, and modular code.
 - To improve problem - solving and logical thinking skills.
 - To gain experience in designing a system that behaves like real - world applications.

MODULES

- Create Credentials Module: Lets the user create
- username, password, and role. Saves them into
- credentials.txt. Login Module: Reads stored
- credentials from file. Validates username and
- password. Loads admin or user menu based on the
- role.
- Admin Menu Module:
- Full control over all operations.
- Accessible only if role = admin.
- User Menu Module:
- Limited access to only view and search options.
- Add Student Module:
- Collects roll, name, and marks.
- Appends the new record to students.txt.
- Display Students Module:
- Reads all records from the file.
- Prints them in table format.
- Search Module:
- Searches file for a roll number.
- Displays record if found.
- Update Module:
- Rewrites file using a temporary file. Updates
- student details when roll number matches.
-

- Delete Module:
- Copies all records except the one to be deleted.
- Safely removes record from file.
- Logout Module:
- Returns control to exit the program.

DATA FLOW DIAGRAM



Explanation

1. Login & Authentication:

The system checks the entered username and password with *credentials.txt* and identifies the user role.

2. Role - Based Menu:

Based on the role (Admin, Staff, Guest), the appropriate menu is shown and the chosen option is passed to the next process.

3. Student Management:

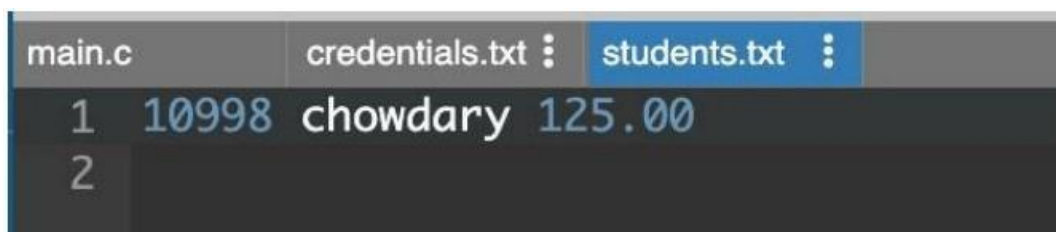
Handles adding, viewing, searching, updating and deleting student records. Updates and deletions use a temporary file for safe rewriting.

4. File Handling:

All student data is stored in *students.txt* , ensuring permanent storage and reliable read/write operations.

CODE & OUTPUTS

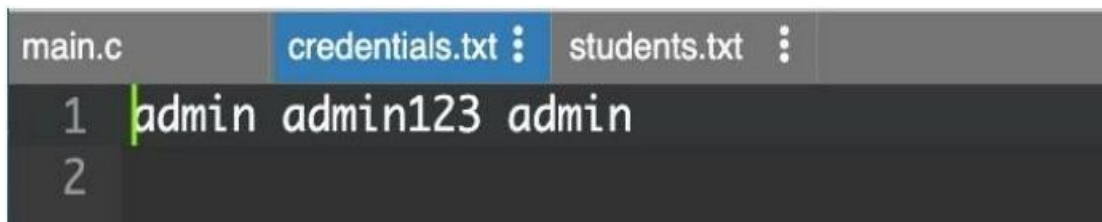
➤ **students.txt**



The screenshot shows a code editor with three tabs: 'main.c', 'credentials.txt', and 'students.txt'. The 'students.txt' tab is active and displays the following text:

```
1 10998 chowdary 125.00
2
```

➤ **Credentials.txt**



The screenshot shows a code editor with three tabs: 'main.c', 'credentials.txt', and 'students.txt'. The 'credentials.txt' tab is active and displays the following text:

```
1 admin admin123 admin
2
```


➤ LOGIN & ADMIN MENU

```
void adminMenu() {
    int choice;
    while (1) {
        printf("\n==== ADMIN MENU =====\n");
        printf("1. Add Student\n");
        printf("2. Display Students\n");
        printf("3. Search Student\n");
        printf("4. Update Student\n");
        printf("5. Delete Student\n");
        printf("6. Logout\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: printf("Logging out...\n"); return;
            default: printf("Invalid choice\n");
        }
    }
}
```

```
===== LOGIN SCREEN =====
Enter username: admin
Enter password: admin123

===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: █
```

After login, the Admin Menu appears, allowing the admin to add, view, search, update, or delete student records.

➤ ADD STUDENT

```
// ADD STUDENT
void addStudent() {
    FILE *fp = fopen(STUDENT_FILE, "a");
    struct Student st;

    if (!fp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll No: ");
    scanf("%d", &st.roll);

    printf("Enter Name: ");
    scanf("%s", st.name);
    printf("Enter Marks: ");
    scanf("%f", &st.marks);

    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);

    printf("Student Added Successfully!\n");
}
```

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter roll number: 10999
Enter name: santhosh
Enter marks: 100
Student added successfully!
```

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter roll number: 10843
Enter name: harshith
Enter marks: 150
Student added successfully!
```

This screen allows the admin to add a new student record by entering the roll number, name, and marks. After submission, the record is successfully stored in the system.

➤ **DISPLAY STUDENT**

```
174 // DISPLAY STUDENTS
175 void displayStudents() {
176     FILE *fp = fopen(STUDENT_FILE, "r");
177     struct Student st;
178
179     if (!fp) {
180         printf("No Student records found\n");
181         return;
182     }
183
184     printf("\nRoll\tName\tMarks\n");
185     printf("-----\n");
186
187     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
188         printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
189     }
190     fclose(fp);
191 }
192
```

===== ADMIN MENU =====

1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout

Enter choice: 2

Roll	Name	Marks

10999	santhosh	100.00
10843	harshith	150.00

This output shows the list of all stored student records. It displays roll numbers, names, and marks in a structured table format.

➤ SEARCH STUDENT

```
193 // SEARCH STUDENT
194 void searchStudent() {
195     FILE *fp = fopen(STUDENT_FILE, "r");
196     struct Student st;
197     int roll, found = 0;
198
199     if (!fp) {
200         printf("Error opening file\n");
201         return;
202     }
203
204     printf("Enter Roll to search: ");
205     scanf("%d", &roll);
206
207     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
208         if (st.roll == roll) {
209             printf("Record found\n");
210             printf("Roll: %d\nName: %s\nMarks: %.2f\n", st.roll, st.name, st.marks);
211             found = 1;
212             break;
213         }
214     }
215
216     if (!found)
217         printf("Record not found\n");
218
219     fclose(fp);
220 }
```

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 3
Enter roll number to search: 10998

Found: 10998 santhosh 100.00
```

This screen lets the user search for a student using the roll number. If the record exists, the student's details are displayed.

➤ UPDATE STUDENT

```
222 // UPDATE STUDENT
223 void updateStudent() {
224     FILE *fp = fopen(STUDENT_FILE, "r");
225     FILE *temp = fopen("temp.txt", "w");
226     struct Student st;
227     int roll, found = 0;
228
229     if (!fp || !temp) {
230         printf("Error opening file\n");
231         return;
232     }
233
234     printf("Enter Roll to update: ");
235     scanf("%d", &roll);
236
237     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
238         if (st.roll == roll) {
239             printf("Enter new Name: ");
240             scanf("%s", st.name);
241             printf("Enter new Marks: ");
242             scanf("%f", &st.marks);
243             found = 1;
244         }
245         fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
246     }
247
248     fclose(fp);
249     fclose(temp);
250
251     remove(STUDENT_FILE);
252     rename("temp.txt", STUDENT_FILE);
253
254     if (found)
255         printf("Record Updated Successfully!\n");
256     else
257         printf("Record not found\n");
258 }
259
```

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 4
Enter roll number to update: 10998
Enter new name: chowdary
Enter new marks: 125
Record updated!
```


The update screen enables modification of an existing student's details. The admin can update the student's name and marks after entering the roll number.

➤ **DELETE STUDENT**

```
// DELETE STUDENT
void deleteStudent() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if (!fp || !temp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll to delete: ");
    scanf("%d", &roll);

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        if (st.roll == roll) {
            found = 1;
            continue;
        }
        fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if (found)
        printf("Record Deleted Successfully!\n");
    else
        printf("Record not found\n");
}
```

```
===== ADMIN MENU =====  
1. Add Student  
2. Display Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Logout  
Enter choice: 5  
Enter roll number to delete: 1032  
Record deleted!
```

This output shows the result of deleting a student record. The system removes the matching roll number from the database and confirms the action.

FUTURE ENHANCEMENT

1. Add password encryption for better security.
2. Add more student details like phone number, email, and department.
3. Allow sorting of records by name, roll, or marks.
4. Add a GUI version using Python, Java, or C++.
5. Migrate from text files to SQL database for faster performance.
6. Add backup and restore options.
7. Generate report cards automatically.
8. Add option to export data to Excel or PDF.
9. Improve search feature with more filters.

CONCLUSION

1. This project helped me understand how programming is used in real -life data management.
2. I learned how to store and retrieve information using files.
3. I also understood how different modules in a program work together.
4. The login system taught me the basics of authentication and user roles.
5. This system is simple but covers all essential record operations.
6. Developing SRMS improved my confidence in C programming.
7. It helped me practice writing clean and structured code.
8. The project also taught me the importance of proper testing.
9. Overall, the SRMS meets all the goals I planned and works effectively.
10. It is a good foundation for creating bigger management systems in the future.